Case Study: Social Media Feed Sorting System Using Merge Sort

NAME: G Durga Surya Prasad

Register no: 192425158

Course code: CSA0609

Title

Social Media Feed Sorting System Using Merge Sort

Description of the Real-World Problem

Social media platforms handle millions of posts every day. A user's feed must present posts in a meaningful order based on factors such as timestamp, relevance, engagement score, or a combined ranking value. The sorting process should be efficient because a large number of posts may need to be processed while maintaining the original order of posts having equal priority.

Merge Sort is suitable for this problem because it follows the Divide and Conquer approach, guarantees O(n log n) time complexity in the best, average, and worst cases, and is a stable sorting algorithm. Stability is important when posts have equal sorting keys because their original relative order can be preserved.

Problem Statement

Design an efficient sorting mechanism for a social media feed using Merge Sort. The system should:

- Sort a large number of posts efficiently.
- Arrange posts according to timestamp and relevance.
- Maintain stability when two posts have equal sorting values.
- Support large-scale data processing.
- Handle challenges caused by frequent real-time feed updates.
- Provide predictable O(n log n) sorting performance.

Algorithm Used

Merge Sort (Divide and Conquer Algorithm)

Merge Sort repeatedly divides the list into smaller sublists until each sublist contains one element. The sorted sublists are then merged while maintaining the required order.

Steps

1. Divide the list into two halves.
2. Recursively divide each half until single-element lists are obtained.
3. Compare elements from the two sorted halves.
4. Merge them into one sorted list in the required order.
5. Continue until the complete list is sorted.

Stability in Social Media Feed Sorting

Merge Sort is a stable sorting algorithm because equal-key elements can maintain their original relative order during the merging process.

For example, if two posts have the same relevance score, the post that appeared earlier in the original sequence can remain before the later post.

This is useful when the feed uses multiple sorting criteria. A stable sort helps preserve the previous ordering when the current sorting key is equal.

Divide and Conquer Mechanism

The social media feed is divided into smaller parts before sorting. Each part is sorted independently, and the sorted parts are merged to create the final ordered feed.

Example

Initial posts:

[85, 42, 96, 33, 77, 68, 90]

Divide

[85, 42, 96] [33, 77, 68, 90]

Divide Again

[85] [42, 96] [33, 77] [68, 90]

The process continues until single elements are obtained.

Merge

[42, 85, 96] [33, 68, 77, 90]

Final Sorted Result

[33, 42, 68, 77, 85, 90, 96]

If the social media feed displays the highest relevance first:

[96, 90, 85, 77, 68, 42, 33]

Handling Timestamp and Relevance

A social media post can contain fields such as:

- Post ID
- Timestamp
- Relevance score
- Engagement score

The sorting system can use multiple criteria.

For example:

- Primary key: Relevance score
- Secondary key: Timestamp
- Posts with equal relevance can be ordered according to their timestamp.

Stable Merge Sort helps preserve the original order of posts when their sorting values are equal.

Challenges in Real-Time Updates

Social media platforms continuously receive new posts and updates.

The major challenges are:

- New posts may arrive continuously.
- Existing posts may receive new relevance or engagement scores.
- Sorting millions of posts after every update can be expensive.
- The system must provide low response time to users.
- Memory usage becomes important for large datasets.

- The system must scale when the number of users and posts increases.

Merge Sort provides predictable O(n log n) sorting performance. However, a real-time system should avoid sorting the entire dataset after every small update.

A practical system can sort only affected feed batches or partitions and then merge the sorted results.

Scalability Considerations

For handling millions of posts:

- Large feeds can be divided into independent batches.
- Different batches can be processed separately.
- Sorted batches can be merged later.
- Data can be processed in parallel.
- External or distributed sorting can be used for very large datasets.
- Stable merging helps maintain consistent ordering.

This makes Merge Sort suitable for large-scale feed processing.

Manual Execution – Step-by-Step Process

Consider the following relevance scores:

[85, 42, 96, 33, 77, 68, 90]

Step 1: Divide the Array

Left: [85, 42, 96]

Right: [33, 77, 68, 90]

Step 2: Continue Dividing

[85] [42, 96] [33, 77] [68, 90]

Continue until every element is separated:

[85] [42] [96] [33] [77] [68] [90]

Step 3: Merge Small Parts

[42] + [96] → [42, 96]

[33] + [77] → [33, 77]

[68] + [90] → [68, 90]

Step 4: Merge Larger Parts

[85] + [42, 96] → [42, 85, 96]

[33, 77] + [68, 90] → [33, 68, 77, 90]

Step 5: Final Merge

[42, 85, 96] + [33, 68, 77, 90]

Final result:

[33, 42, 68, 77, 85, 90, 96]

Example of Stable Sorting

Suppose two posts have the same relevance:

Post A → Relevance 90 → Timestamp 10:00

Post B → Relevance 90 → Timestamp 10:05

During a stable merge, Post A remains before Post B when their primary sorting key is equal.

This prevents unnecessary changes in the user's feed ordering.

Complexity Analysis

Case	Time Complexity	Space Complexity
Best Case	O(n log n)	O(n)
Average Case	O(n log n)	O(n)
Worst Case	O(n log n)	O(n)

Explanation

Best Case

- The array is repeatedly divided into smaller parts.
- The merging process still performs the required work.
- The overall time complexity is O(n log n).

Average Case

- The array is divided into approximately equal halves.
- Each level of recursion performs O(n) merging work.
- There are approximately log n levels.
- Therefore, the total time complexity is O(n log n).

Worst Case

Unlike Quick Sort, Merge Sort does not become $O(n^2)$ because of a poor pivot.

- The array is always divided into balanced halves.
- The merging work remains O(n) at each level.
- Therefore, the worst-case time complexity is O(n log n).

Space Complexity

Standard Merge Sort requires O(n) additional memory for temporary arrays during merging.

Comparison with Other Algorithms

Algorithm	Best Case	Average Case	Worst Case	Stable	Space
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	No	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No	O(1)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)

Algorithm	Best Case	Average Case	Worst Case	Stable	Space
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	O(1)

Why Merge Sort is Appropriate

Merge Sort is appropriate for a social media feed sorting system because:

- It guarantees O(n log n) time complexity.
- It is a stable sorting algorithm.
- It is efficient for large datasets.
- It uses the Divide and Conquer technique.
- It provides predictable performance.
- It can be adapted to sort records using multiple fields.
- It can be used for batch, external, and distributed sorting.
- It avoids the $O(n^2)$ worst-case problem of Quick Sort.

Applications

Merge Sort can be used in:

- Social media news feeds
- Post ranking systems
- Search result ordering
- Notification prioritization
- Content recommendation systems
- News aggregation systems
- E-commerce recommendation feeds
- Large-scale analytics systems

Advantages

- Guaranteed O(n log n) time complexity.
- Stable sorting algorithm.
- Efficient for large datasets.
- Uses Divide and Conquer.
- Suitable for large-scale data processing.
- Predictable performance.
- Can be used for external and distributed sorting.
- Maintains the order of equal-priority elements.

Limitations

- Requires O(n) additional memory.
- Sorting the complete feed after every real-time update can be expensive.
- More memory is required compared with in-place sorting algorithms.
- A separate strategy is required for handling continuous real-time updates.

Suitable Solution Design

A suitable social media feed system can divide posts into manageable batches. Each batch can be sorted using stable Merge Sort according to relevance and timestamp.

When new posts arrive or existing posts are updated, only the affected batches can be processed whenever possible. The sorted batches can then be merged to generate the final feed.

For very large-scale systems, different partitions can be processed independently and the sorted results can be merged. This improves scalability and follows the Divide and Conquer principle.

Pseudocode

START

Read the list of social media posts

If the list contains zero or one post

Return the list

Divide the list into two halves

Recursively apply Merge Sort to the left half

Recursively apply Merge Sort to the right half

Merge the two sorted halves

Compare relevance scores

If relevance is equal

Compare timestamps

Preserve original order for equal keys

Return the merged sorted list

STOP

Python Implementation

```
def merge_sort(arr):
    # Stop recursion for one-element lists
    if len(arr) <= 1:
```

```
    return arr

# Divide the array into two halves
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])

# Create a list to store the merged result
result = []
i = 0
j = 0

# Merge the two sorted halves
while i < len(left) and j < len(right):
    # Use <= to maintain stability
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

# Add remaining elements
result.extend(left[i:])
result.extend(right[j:])

return result
```

```
        # Add remaining elements
        result.extend(left[i:])
        result.extend(right[j:])

        return result


    # Example input
    scores = [85, 42, 96, 33, 77, 68, 90]

    print("Original Scores:", scores)
    print("Sorted Scores:", merge_sort(scores))
```

```
Original Scores: [85, 42, 96, 33, 77, 68, 90]
Sorted Scores: [33, 42, 68, 77, 85, 90, 96]
```

Conclusion

A social media platform requires an efficient and stable sorting mechanism to organize millions of posts according to relevance and timestamp. Merge Sort is an appropriate choice because it uses the Divide and Conquer strategy and provides O(n log n) time complexity in the best, average, and worst cases.

Its stability is especially useful because posts with equal ranking values can retain their original relative order. However, real-time updates and large-scale data create challenges because sorting the entire dataset after every update can be expensive.

A scalable solution can process posts in batches, sort affected partitions, and merge the sorted results. Therefore, Merge Sort provides predictable performance, stable ordering, and scalability, making it suitable for large-scale social media feed sorting systems.